Complete Guide to How Web Applications Work

This document explains how modern web applications work from the moment a user types a URL in a browser until the webpage appears on the screen. It also covers HTTP vs HTTPS, browser architecture, HTTP methods, REST APIs, and the typical architecture used in modern applications such as Angular frontends connected to .NET APIs and SQL Server databases.

1. User Enters a URL in the Browser

When a user enters a URL such as https://google.com in a browser, the browser first parses the URL. It identifies the protocol (HTTPS), the domain name (google.com), and the resource path. The browser must then locate the server hosting the website.

2. DNS Lookup

Computers communicate using IP addresses. Therefore the browser must convert the domain name into an IP address using the Domain Name System (DNS). The lookup process usually checks: • Browser cache • Operating system cache • Router cache • ISP DNS servers • Root DNS servers Eventually the correct IP address of the web server is returned.

3. TCP Connection

After obtaining the server IP address, the browser establishes a TCP connection. TCP ensures reliable communication between the client and server. The TCP three■way handshake includes: 1. SYN (client requests connection) 2. SYN■ACK (server acknowledges) 3. ACK (client confirms)

4. TLS Handshake (for HTTPS)

If the site uses HTTPS, a TLS handshake occurs before data exchange. The server sends its SSL certificate to prove its identity. The browser verifies the certificate and negotiates encryption keys. After this process, communication becomes encrypted.

5. HTTP Request

Once the connection is established, the browser sends an HTTP request to the server. Example request structure: Method: GET Path: / Headers: Host, Cookies, User-Agent Body: Optional (used mainly with POST requests)

6. Server Processing

The server receives the request and processes it using backend logic. This logic may run in frameworks such as ASP.NET, Node.js, Django, or Spring. Often the server queries a database to retrieve information before generating the response.

7. HTTP Response

The server sends back an HTTP response containing: • Status code (200 OK, 404 Not Found, 500 Internal Error) • Headers • HTML document • CSS files • JavaScript files • Images and media

8. Browser Rendering

The browser parses HTML to construct the DOM (Document Object Model). CSS styles are applied to create layout and visual design. JavaScript executes logic and enables interactive behavior. The browser then paints pixels on the screen to display the webpage.

Difference Between HTTP and HTTPS

Feature	HTTP	HTTPS
Full Form	HyperText Transfer Protocol	HyperText Transfer Protocol Secure
Security	No encryption	Encrypted using TLS/SSL
Port	80	443
Data Protection	Data can be intercepted	Encrypted and secure
Use Cases	Old systems or internal networks	Modern websites, banking, logins
SEO Ranking	Lower	Preferred by search engines

Browser Architecture

- User Interface – address bar, navigation buttons, bookmarks.
- Browser Engine – coordinates actions between UI and rendering engine.
- Rendering Engine – parses HTML/CSS and builds the visual layout.
- Networking Layer – handles HTTP/HTTPS requests.
- JavaScript Engine – executes JavaScript code.
- Data Storage – cookies, local storage, session storage.

Common HTTP Methods

Method	Purpose
GET	Retrieve data from server
POST	Send new data to server
PUT	Update existing resource
DELETE	Remove resource
PATCH	Partial update

REST API Concept

REST (Representational State Transfer) is a common architectural style for building web APIs. REST APIs expose resources using URLs and use HTTP methods to operate on them. Example endpoints: GET /api/users POST /api/users GET /api/users/1 DELETE /api/users/1

Modern Web Application Architecture

Most modern applications follow a three■layer architecture: Frontend Layer: Frameworks such as Angular or React run in the browser and handle UI rendering. Backend Layer: APIs built with technologies like ASP.NET Core process requests and implement business logic. Database Layer: Databases such as SQL Server store persistent data. Typical request flow: Browser (Angular UI) ↓ HTTP Request ↓ ASP.NET Core Web API ↓ SQL Server Database ↓ Response back to Browser